

Money Movement Application

Requirements Document — PSTU IT Carnival 2025 Hackathon

Challenge	Money Movement Application
Event	PSTU IT Carnival 2025 — 29 August 2026
Team Size	2 members
Stack	Django (backend) + React (frontend), PostgreSQL
Duration	9:00 AM – 3:00 PM (6 hours)

This document defines the functional and non-functional requirements for the Money Movement Application, along with a prioritized backlog of secondary features. Core scope covers account management, sending and requesting money, and transaction history, built on a ledger designed for correctness under concurrent, high-growth usage.

1. Functional Requirements

1.1 Account & Identity

- **FR-1** — A user can register a new account with a unique username and password.
- **FR-2** — On registration, the system automatically credits the new account with a starting balance of BDT 100,000.
- **FR-3** — A registered user can log in and authenticate.
- **FR-4** — A logged-in user can log out, invalidating their session/token.

1.2 Money Movement (Core)

- **FR-5** — A user can view their current account balance.
- **FR-6** — A user can send money directly to another user by exact username, with an amount and an optional note. The transfer executes immediately.
- **FR-7** — A user can request money from another user by exact username, with an amount and an optional note. This creates a pending request rather than moving money immediately.
- **FR-8** — The payer (recipient of a request) can approve a pending request, which triggers a money transfer identical in effect to a direct send.
- **FR-9** — The payer can decline a pending request; no money moves and the request is marked declined.
- **FR-10** — The requester can cancel their own pending outgoing request before it is approved or declined.
- **FR-11** — Both "send" and "approved request" money movements are executed through the same single transfer operation in the system, ensuring one consistent, trustworthy code path regardless of how a transfer was triggered.

1.3 Transaction Records

- **FR-12** — A user can view a history of all their past transactions (sent and received).
- **FR-13** — A user can view the detail of a single transaction: counterparty, amount, timestamp, status, and note.

- **FR-14** — A user can view their pending requests, separated into incoming (awaiting their action) and outgoing (awaiting the other party's action).
- **FR-15** — Every transaction has a visible status: pending, completed, or failed.

1.4 Validation & Error Handling

- **FR-16** — The system rejects a transfer/request if the sender has insufficient balance, with a clear error message.
- **FR-17** — The system rejects zero, negative, or non-numeric amounts.
- **FR-18** — The system rejects a user sending or requesting money from themselves.
- **FR-19** — The system rejects a transfer or request addressed to a username that does not exist, with a clear error message.
- **FR-20** — Duplicate submission of the same action (e.g. double-tapping "Send") does not result in the money being moved twice.

1.5 Access Control

- **FR-21** — A user can only view their own balance and transaction history, never another user's.
- **FR-22** — A user can only approve or decline requests addressed to them.
- **FR-23** — A user can only cancel requests they themselves created.

2. Non-Functional Requirements

2.1 Correctness & Data Integrity

- **NFR-1 (Atomicity)** — A transfer either fully completes (both balances updated) or does not happen at all; no partial state is ever persisted.
- **NFR-2 (Consistency)** — At any point in time, the sum of all account balances equals the total money issued into the system ($\text{BDT } 100,000 \times \text{number of registered users}$). This invariant can be verified directly with a single aggregate query.
- **NFR-3 (No negative balances)** — No account balance can go below zero, under any load or timing condition.
- **NFR-4 (Durability)** — Once a transaction is marked completed, it persists even in the event of a server crash or restart.

2.2 Concurrency

- **NFR-5** — The system correctly handles simultaneous transfer requests against the same account without double-spending or lost updates, using row-level locking on account records during a transfer.
- **NFR-6** — Concurrent transfers between the same pair of users in opposite directions do not cause a deadlock (achieved by acquiring account locks in a consistent, sorted order).

2.3 Reliability

- **NFR-7 (Idempotency)** — Retried or duplicate requests (e.g. from a network glitch or repeated tap) do not cause the money movement to execute more than once, enforced via a unique idempotency key per action.
- **NFR-8** — A failed transaction leaves the system in a clean, recoverable state and is never left stuck in "pending" indefinitely.

2.4 Security

- **NFR-9** — Passwords are stored hashed, never in plaintext.
- **NFR-10** — Authentication is required for all balance and money-movement endpoints.
- **NFR-11** — A user cannot access or modify another user's account data (enforced server-side, not just hidden in the UI).
- **NFR-12** — All endpoint input is validated server-side against malformed or malicious input.

2.5 Performance & Scalability

- **NFR-13** — Username and foreign-key columns are indexed to keep lookups fast as the user base grows.
- **NFR-14** — Transaction history is served paginated, not loaded in full on every request.
- **NFR-15** — The backend is stateless, so additional server instances can be added behind a load balancer without code changes.
- **NFR-16** — PostgreSQL is used specifically for its row-level locking and multi-user concurrency support, in place of a file-based database such as SQLite.

2.6 Usability

- **NFR-17** — Error messages shown to the user are clear and actionable, never raw stack traces or generic failures.
- **NFR-18** — The user always has visibility into the current status of a transaction — no money movement happens as an invisible "black box."

2.7 Auditability

- **NFR-19** — Every money movement is permanently recorded in an append-only transaction ledger; records are never hard-deleted, preserving a full audit trail.

3. Secondary Feature (Time-Permitting)

If time remains after the core functional and non-functional requirements are complete, the team will implement a fraud/anomaly detection layer on top of the transaction system. A trained ML model is not feasible within the available time, since there is no real transaction history to train on. Instead, this is implemented as a rule-based detection engine — the same category of problem, explainable and demoable within the time budget.

3.1 Rule-Based Fraud / Anomaly Detection Engine

- Flags a transaction as suspicious without blocking it, based on simple heuristics evaluated at transfer time.
- **Large relative to balance** — transaction amount exceeds a set percentage of the sender's current balance.
- **High frequency** — the same sender makes several transfers within a short time window.
- **Round-trip pattern** — money sent from A to B is immediately sent back from B to A.
- **Unusual amount for the user** — transaction amount deviates significantly from that user's own historical average.
- Flagged transactions are stored with a risk flag and surfaced in the transaction history / an oversight view, for review rather than automatic rejection.

Prepared for team use — PSTU IT Carnival 2025 Hackathon, Money Movement Application challenge.